# Parallelism on GPU

Parallelism TP

October 14, 2026

# GPU: OpenMP (1)

OpenMP can run parallel code on the GPU:

```
#pragma omp parallel for

// .. becomes ..

#pragma omp target teams distribute parallel for
```

You must specify which data needs to be transferred between the CPU and the GPU.

# GPU: OpenMP (2)

Different mappings are possible:

- `map(to:...)` transfers from the host to the device (CPU →GPU).
- `map(from:...)` transfers from the device to the host (GPU →CPU).
- `map(tofrom:...)` transfers from the host to the device, then from the device back to the host (CPU →GPU →CPU).
- `map(alloc:...)` allocates on the device (GPU), without any transfer.

Each mapping takes a range as parameter: `map(alloc: A[0:N])`

Requires converting `std::vector<>` to pointers.

# GPU: OpenMP (3)

Example:

```
const std::vector<int> vec_a{ 1, 2, 3, 4 };
const std::vector<int> vec_b{ 5, 6, 7, 8 };
const size_t n = vec_a.size();
std::vector<int> vec_c(n, 0);

const auto* const a = vec_a.data();
const auto* const b = vec_b.data();
auto* const c = vec_c.data();

#pragma omp target teams distribute parallel for \
    map(to: a[0:n], b[0:n]) \
    map(from: c[0:n])
for(size_t i = 0; i < n; ++i) {
    c[i] = a[i] + b[i];
}
```

# GPU: OpenMP (4)

The target is specified at compile time:

```
g++ -fopenmp -foffload=nvptx-none -O3 main.cpp -o main
# or
clang++ -fopenmp -fopenmp-targets=nvptx64-nvidia-cuda \
    --offload-arch=sm_75 -O3 main.cpp -o main
# or
nvc++ -mp=gpu -fopenmp -gpu=cc75 -O3 main.cpp -o main
```

**Note**

If the target is not specified, the parallelism runs on the host.

# GPU: Standard (1)

Goal: standard code. Memory transfers must be handled by the compiler.

For Nvidia: the `nvc++` compiler supports offloading standard algorithms to the GPU with the `-stdpar=gpu` option.

Some constraints on the parallel code:

- The accessed memory must be on the heap
- No function pointers
- No polymorphism
- ...

# GPU: Standard (2)

Example:

```
const std::vector<int> vec_a{ 1, 2, 3, 4 };
const std::vector<int> vec_b{ 5, 6, 7, 8 };
const size_t n = vec_a.size();
std::vector<int> vec_c(n, 0);

const auto* const a = vec_a.data();
const auto* const b = vec_b.data();
const auto* const c = vec_c.data();

std::for_each(std::execution::par_unseq,
    vec_c.begin(), vec_c.end(), [=](int& x) {
        const size_t i = &x - c; // Get index
        x = a[i] + b[i]; // Use ptr
    }
);
```

# GPU: Standard (3)

The offloading is specified at compile time:

```
nvc++ -stdpar=gpu -O3 main.cpp -o main
```